

第5章: 网络层控制平面



学习目标: 理解网络控制平面原理

- 传统路由算法
- SDN控制器
- ICMP协议
- 网络管理

在因特网的实现:

- OSPF, BGP, OpenFlow, ICMP, SNMP

5-1

第5章: 网络层控制平面



5.1 简介

5.2 路由协议

- 链路状态
- 距离向量

5.3 自治系统内的路由:

OSPF

5.4 ISP之间的路由: BGP

5.5 SDN控制平面

5.6 ICMP协议

5.7 SNMP协议

5-2

网络层功能

回顾：网络层两个功能：

- 转发：将分组从路由器的输入端口 数据平面
转发到恰当的输出口
- 路由：决定分组从源主机到目的 控制平面
主机的传输路径

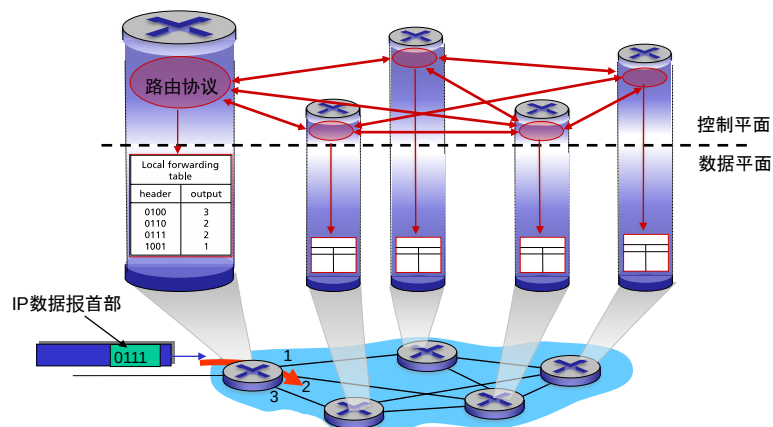
控制平面的方法：

- 每路由器控制（传统方法）
- 逻辑集中式控制（软件定义网络SDN）

5-3

控制平面：每路由器控制

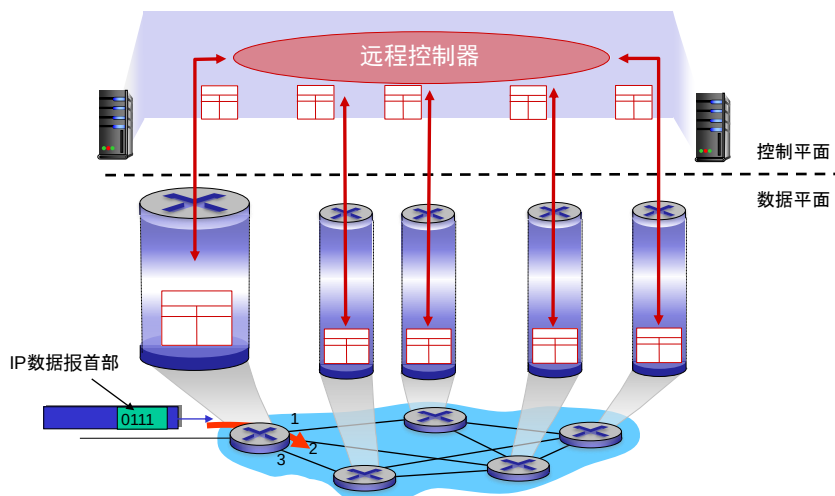
在各路由器上配置、运行路由协议，生成转发表



5-4

控制平面：逻辑集中式控制

远程控制器计算、分发流表



第5章: 网络层控制平面



5.1 简介

5.5 SDN控制平面

5.2 路由协议

5.6 ICMP协议

■ 链路状态

5.7 SNMP协议

■ 距离向量

5.3 自治系统内的路由:

OSPF

5.4 ISP之间的路由: BGP

5-6

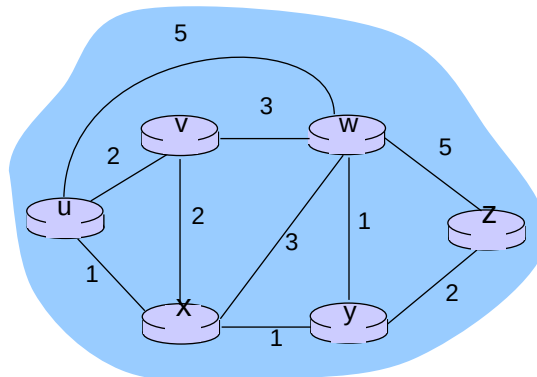
路由协议

目标: 找到从源主机到目的主机的好路径

- 路径: 经过的一系列路由器
- 好路径: 代价小: 距离短、速度快、拥塞少
- 网络的10大问题之一!

5-7

网络拓扑图



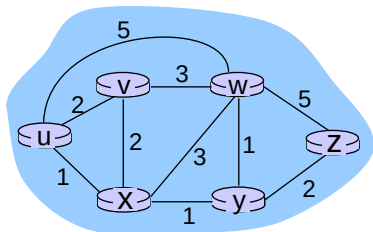
$G = (N, E)$

$N = \text{路由器集合} = \{ u, v, w, x, y, z \}$

$E = \text{路由器间链路集合} = \{ (u,v), (u,x), (v,x), (v,w), (x,w), (x,y), (w,y), (w,z), (y,z) \}$

5-8

链路代价



费用（代价）：距离、时延、拥塞等

$c(x,y)$ = 链路(x,y)的费用

e.g., $c(w,z) = 5$

路径表示：路径所经过的节点序列

e.g., (u,x,y,z)

路径费用：沿途所有边的费用之和

e.g., $c(u,x) + c(x,y) + c(y,z)$

Q: u和z之间的最小代价路径是?
路由算法: 寻找最小代价路径

最低费用路径：路径费用最小的路径

最短路径：源和目的地之间经过链路最少的路径，即所有链路的费用都相同的情况

5-9

路由算法分类

集中式vs分散式

集中式:

- 各路由器知道整个网络拓扑和链路代价
- 链路状态LS算法

分散式:

- 各路由器只知道邻居路由器及其链路代价
- 邻居间周期性交换信息
- 距离向量DV算法

静态vs动态

静态:

- 路由随时间缓慢变化，通常人工配置

动态:

- 路由变化快
 - 周期性更新
 - 响应网络变化

5-10

第5章: 网络层控制平面



5.1 简介

5.5 SDN控制平面

5.2 路由协议

5.6 ICMP协议

■ 链路状态

5.7 SNMP协议

■ 距离向量

5.3 自治系统内的路由:

OSPF

5.4 ISP之间的路由: BGP

5-11

链路状态(LS)算法

Dijkstra算法

- 各节点知道整个网络拓扑及链路代价

- 节点广播本地链路状态（与邻居的链路代价），后续有变化就广播

- 计算自己(源节点)到**所有节点**的最小代价路径

- 以此构建自己的转发表

- 迭代算法: 由近至远，每次迭代算出源节点到一个节点的最小代价路径

变量:

- $c(x,y)$: (x, y) 链路费用，如果 x 与 y 非直连, $c(x, y) = \infty$

- $D(v)$: 从源节点到 v 的当前路径费用

- $p(v)$: 源节点到 v 的路径上, v 的前一个节点

- N' : 已算出最低费用路径的节点集合



$$c(m, n) = 1, c(m, e) = \infty$$

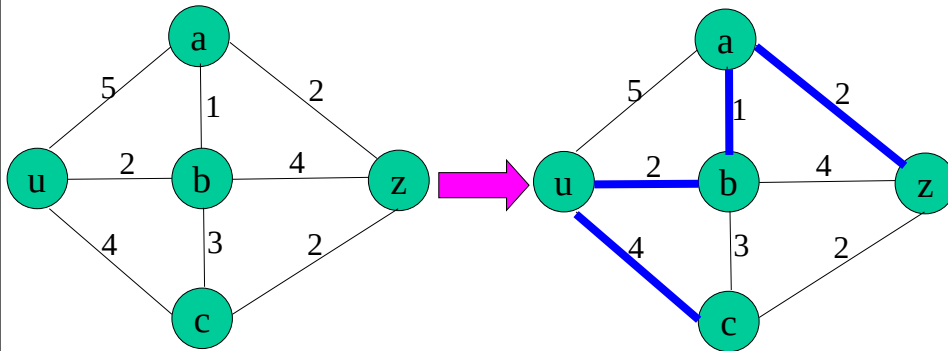
$$D(n) = 2$$

$$p(n) = m$$

$$N' = \{u, m, n\}$$

5-12

Dijkstra算法: 思考

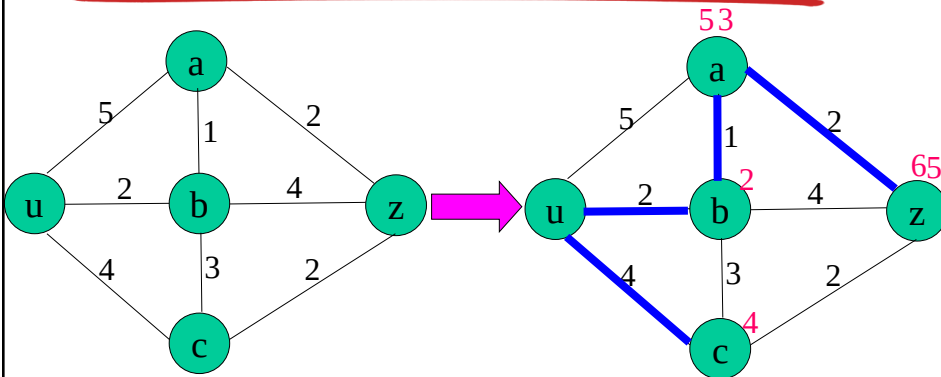


目标：找出从源节点u到所有其他节点的最低费用路径

前提：每个节点都已知网络拓扑和所有链路的费用

5-13

Dijkstra算法: 思路



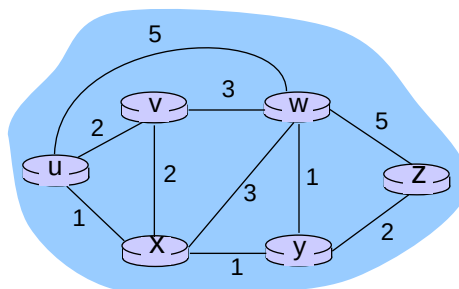
核心思想：

1. 初始化每个节点费用
2. 找出最小费用的节点 (记为a)，更新a的所有不在N' 中的邻居 (记为b) 的费用，
即： $D(b) = \min[D(b), D(a) + c(a, b)]$
3. 在剩下的节点中循环迭代第2步，...

5-14

Dijkstra算法: 例子

步骤	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy	2,u	3,y			4,y
3	uxyv		3,y			4,y
4	uxyvw					4,y
5	uxyvwz					

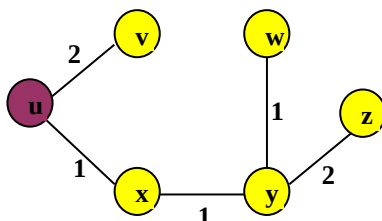


5-15

Dijkstra算法: 例子

u为根的最低费用路径树:

以上表格中5个箭头显示了连接关系



u的转发表:

目的地址	输出链路	目的地址	输出链路
v	(u,v)	v	(u,v)
x	(u,x)	*	(u,x)
y	(u,x)		
w	(u,x)		
z	(u,x)		

注意:

- ❖ 此处是起始节点u的最低费用路径树，即保证从u出发到其他点费用最低；不能保证除节点u外的其他任意两个节点经此路径树费用最低。

思考:

- ❖ 如何获得最短路径?
- ❖ 可能有多条最短路径

5-16

Dijkstra算法

```

1 Initialization:
2   $N' = \{u\}$ 
3  for each node  $v$ 
4    if  $v$  adjacent to  $u$ 
5      then  $D(v) = c(u,v)$ 
6    else  $D(v) = \infty$ 
7
8 Loop
9   find  $w$  not in  $N'$  such that  $D(w)$  is a minimum
10  add  $w$  to  $N'$ 
11  update  $D(v)$  for all  $v$  adjacent to  $w$  and not in  $N'$  :
12     $D(v) = \min(D(v), D(w) + c(w,v))$ 
13    /* new cost to  $v$  is either old cost to  $v$  or known
14       shortest path cost to  $w$  plus cost from  $w$  to  $v$  */
15 until all nodes in  $N'$ 
    
```

5-17

Dijkstra算法

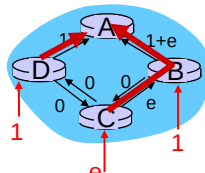
算法复杂度: n 个节点

- 每次迭代: 从不在 N' 的节点中找出开销最小的 w
- $n(n-1)/2$ 次比较: $O(n^2)$
- 采用堆 (基于数组的完全二叉树): $O(n \log n)$

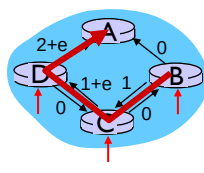
路径振荡问题:

- 假设: 链路代价=链路负载:

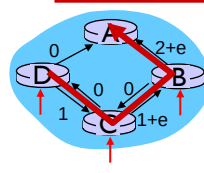
办法: 避免所有的路由器
同时运行LS算法



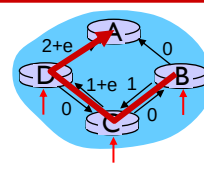
初始路由



B, C发现更好路径,
顺时针



B, C, D发现更好路径,
逆时针



B, C, D发现更好路径,
顺时针

5-18

第5章: 网络层控制平面



- 5.1 简介
- 5.2 路由协议
 - 链路状态
 - 距离向量
- 5.3 自治系统内的路由:
OSPF
- 5.4 ISP之间的路由: BGP
- 5.5 SDN控制平面
- 5.6 ICMP协议
- 5.7 SNMP协议

5-19

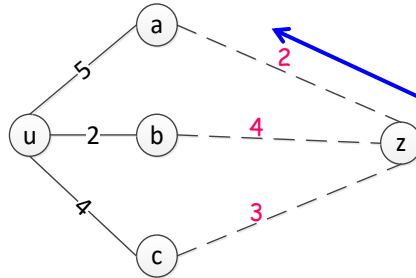
距离向量(DV)算法

是一种异步、迭代、自我终止、分布式的算法

- ✓ 异步: 不要求所有节点相互之间步伐一致地操作
- ✓ 迭代: 计算过程一直持续到邻居之间无更多信息交换为止
- ✓ 自我终结: 算法能自行停止
- ✓ 分布式: 无需全局拓扑, 每个节点只从其直接相连邻居接收信息, 进行计算, 再将结果分发给邻居

5-20

距离向量(DV)算法-思考



目标：找出从源节点u到任何目标节点(如z)的最低费用

前提：每个节点只知道邻居和与邻居的链路费用

核心思想：找出u通过哪个邻居到z费用最低，即：

$$\min\{c(u,a) + d_a(z), c(u,b) + d_b(z), c(u,c) + d_c(z)\}$$

其中 $d_a(z)$, $d_b(z)$, $d_c(z)$ 可以通过同样的方法一直迭代到z的邻居

5-21

距离向量(DV)算法

Bellman-Ford方程：

令

$d_x(y)$: x到y的最小代价

则

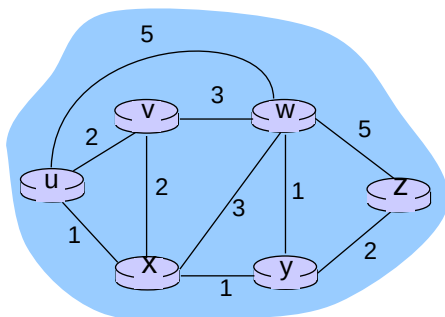
$$d_x(y) = \min_v \{c(x, v) + d_v(y)\}$$

x的每一个邻居v

$\min_v$: 从所有x的邻居到节点y的费用中选取的最小路径费用

5-22

Bellman-Ford方程：例子



已知 $d_v(z) = 5$,
 $d_x(z) = 3$,
 $d_w(z) = 3$

B-F方程：

$$\begin{aligned} d_u(z) &= \min \{ c(u, v) + d_v(z), \\ &\quad c(u, x) + d_x(z), \\ &\quad c(u, w) + d_w(z) \} \\ &= \min \{ 2 + 5, \\ &\quad 1 + 3, \\ &\quad 5 + 3 \} = 4 \end{aligned}$$

u到z，下一步走x最短
 写入u的转发表

5-23

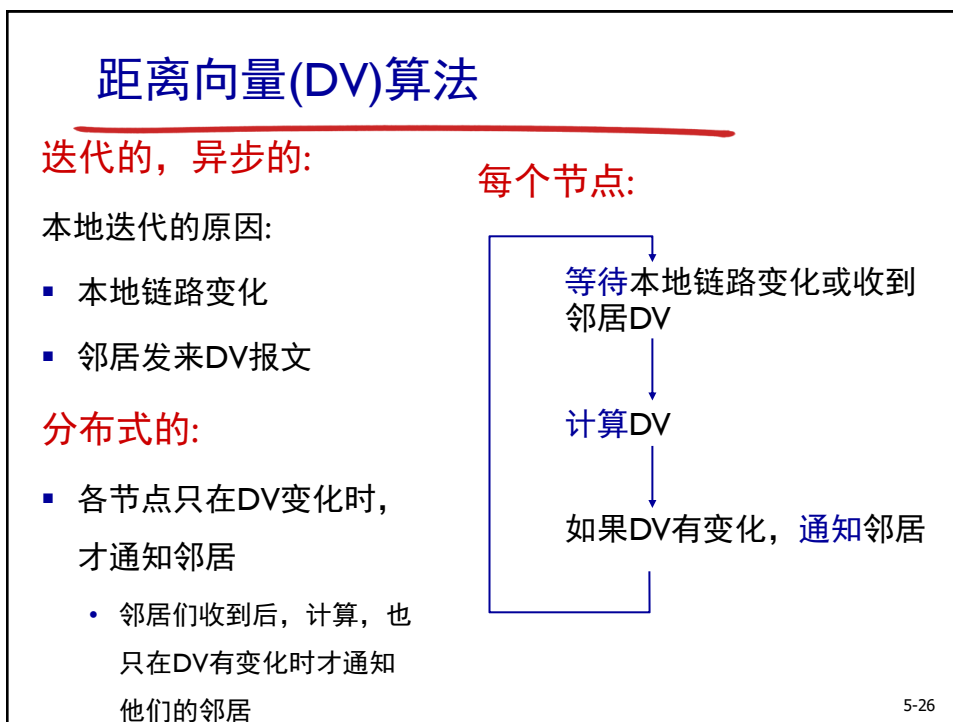
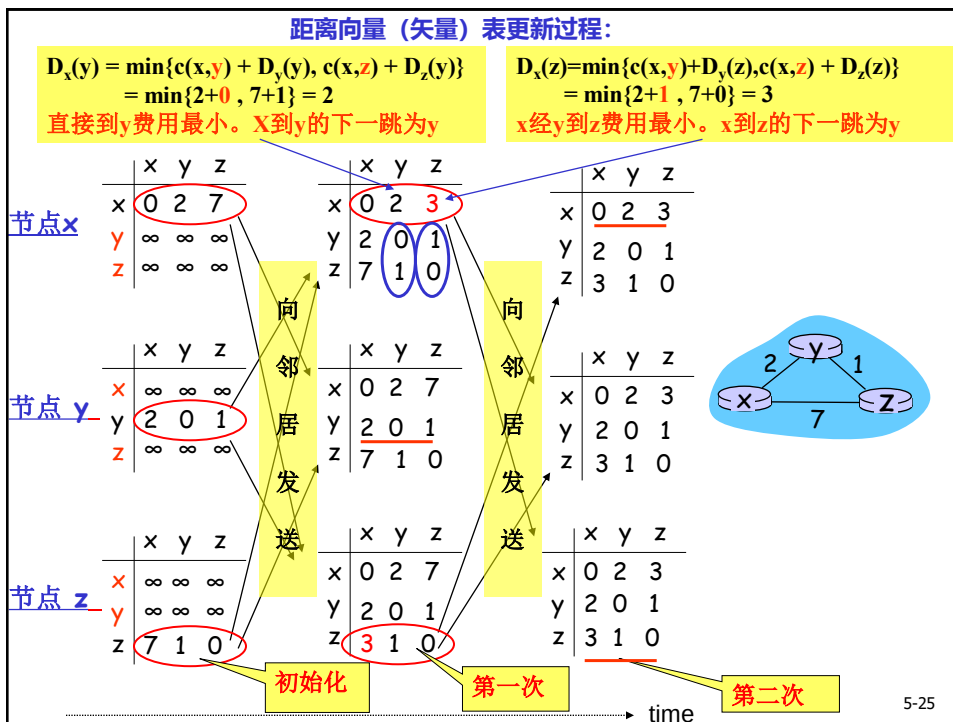
距离向量(DV)算法

原理：

- 假设：
 - $D_x(y)$ = x到y最小代价的估计
 - $D_x = [D_x(y) : y \in N]$ ，距离向量DV，各节点维护一个
- 每个节点有一张**选路表（距离向量表）**
- 当本地链路变化或收到邻居新的DV（更新DV的条件），节点根据B-F方程更新自己的DV：

$$D_x(y) \leftarrow \min_v \{ c(x, v) + D_v(y) \} \quad \text{for each node } y \in N$$
- 如果DV有变化，发送给邻居
- 当DV不再变化，算法静止， $D_x(y)$ 将收敛到 $d_x(y)$

5-24



链路代价变化-好消息

当y与x的链路费用从4变为1，计算y, z到x的最小费用，节点y和z的更新如下：

链路变化前：

$$Dy(x) = 4, Dz(x) = 5$$

t_0 : y 发现本地链路变化，计算并更新DV，
 $Dy(x) = \min\{c(y,x) + Dx(x), c(y,z) + Dz(x)\}$
 $= \min\{1+0, 1+5\} = 1$

下一跳为x，通知邻居z: $Dy(x) = 1$

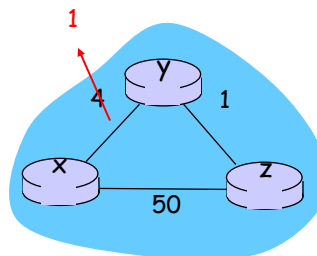
t_1 : z 收到y新的DV后，计算并更新DV，
 $Dz(x) = \min\{c(z,x) + Dx(x), c(z,y) + Dy(x)\}$
 $= \min\{50+0, 1+1\} = 2$

下一跳为y，通知邻居y: $Dz(x) = 2$

t_2 : y 收到z新的DV后，计算，DV无更新，
 $Dy(x) = \min\{c(y,x) + Dx(x), c(y,z) + Dz(x)\}$
 $= \min\{1+0, 1+2\} = 1$

下一跳为x，**不再发送自己的DV**

算法静止



好消息传得快

5-27

链路代价变化-坏消息

当y与x的链路费用从4变为60，计算y, z到x的最小费用，节点y和z的更新如下：

链路变化前：

$$Dy(x) = 4, Dz(x) = 5$$

t_0 : y 发现本地链路变化，计算并更新DV:
 $Dy(x) = \min\{c(y,x) + Dx(x), c(y,z) + Dz(x)\}$
 $= \min\{60+0, 1+5\} = 6$

下一跳为z，通知邻居z, $Dy(x) = 6$

t_1 : z 收到y新的DV后，计算并更新DV:
 $Dz(x) = \min\{c(z,x) + Dx(x), c(z,y) + Dy(x)\}$
 $= \min\{50+0, 1+6\} = 7$

下一跳为y，通知邻居y, $Dz(x) = 7$

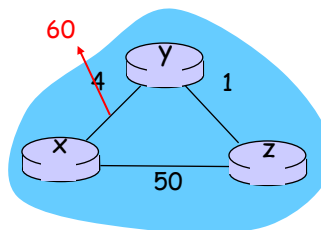
t_2 : y 收到z新的DV后，计算并更新DV:
 $Dy(x) = \min\{c(y,x) + Dx(x), c(y,z) + Dz(x)\}$
 $= \min\{60+0, 1+7\} = 8$

下一跳为z，通知邻居z: $Dy(x) = 8$

.....

上述循环将持续44次迭代，算法才静止

主要问题：产生“选路回环”：为到达x，y通过z选路，z又通过y选路



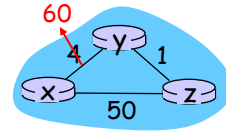
坏消息传得慢

5-28

链路代价变化-毒性逆转

避免“选路回环”现象

毒性逆转法：如果z通过y去往x,则z必须告诉y: 我去往x的代价为无穷 ∞ (y认为z没有到x的路径,就不会试图再经由z选路到x)



链路变化前: z通知y, $Dz(x) = \infty$

t_0 : y发现本地链路变化, 计算并更新DV:

$$Dy(x) = \min\{c(y,x) + Dx(x), c(y,z) + Dz(x)\} \\ = \min\{60+0, 1+\infty\} = 60$$

下一跳为x, 通知邻居z: $Dy(x) = 60$

t_1 : z收到y新的DV后, 计算并更新DV:

$$Dz(x) = \min\{c(z,x) + Dx(x), c(z,y) + Dy(x)\} \\ = \min\{50+0, 1+60\} = 50$$

下一跳为x, 通知邻居y: $Dz(x) = 50$

t_2 : y收到z新的DV后, 计算并更新DV:

$$Dy(x) = \min\{c(y,x) + Dx(x), c(y,z) + Dz(x)\} \\ = \min\{60+0, 1+50\} = 51$$

下一跳为z, 通知邻居z: $Dy(x) = \infty$

t_3 : z收到y新的DV后, 计算, DV无更新:

$$Dz(x) = \min\{c(z,x) + Dx(x), c(z,y) + Dy(x)\} \\ = \min\{50+0, 1+\infty\} = 50$$

下一跳为x, 不再发送自己的DV

算法静止。

毒性逆转法主要问题：三个或更多节点的回路检测不到

5-29

LS算法 vs. DV算法

报文数量

- **LS:** 发送的报文: $O(nE)$, n为节点数, E为链路数

- **DV:** 只在邻居间交换报文
 - 迭代次数不定

健壮性：如果节点故障会如何?

收敛速度

- **LS:** $O(n^2)$
 - 可能路由振荡
- **DV:** 收敛时间不定
 - 可能计数到无穷

LS: 各节点只计算自己的转发表

DV: 节点转发信息扩散给其它节点

- 错误扩散

5-30

第5章: 网络层控制平面



5.1 简介

5.5 SDN控制平面

5.2 路由协议

5.6 ICMP协议

- 链路状态

5.7 SNMP协议

- 距离向量

5.3 自治系统内的路由:

OSPF

5.4 ISP之间的路由: BGP

5-31

路由算法的应用

目前为止:

- 路由器相同 (路由算法)
- 网络扁平

... 但实际:

规模大: 百万条目的地址: 分级管理

- 转发表放不下!
 - 链路被路由报文淹没!
- Internet = 许多ISP网络
 - ISP管理者希望指定路由算法

5-32

因特网的路由算法

自治系统AS (Autonomous Systems) : ISP中的网络,
一级ISP可能是一个或多个AS

内部路由协议

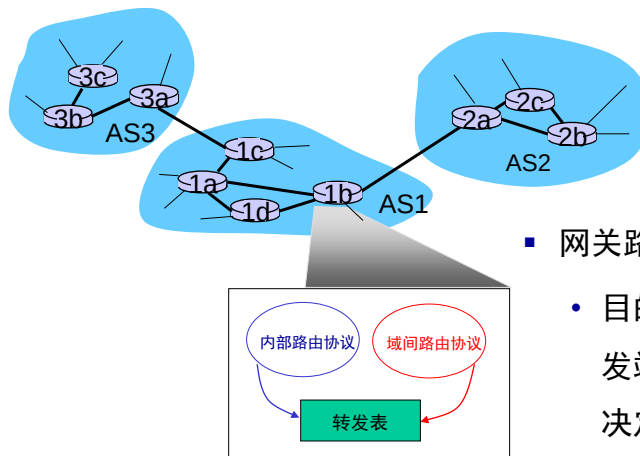
- 在AS内部寻找路由
- 内部路由器运行相同协议
- 不同AS可运行不同协议
- 网关路由器：在AS的边缘，连接其他AS

域间路由协议

- 在AS之间寻找路由
- 网关路由器：运行两类协议

5-33

网关路由器



- 网关路由器的转发表：
 - 目的地址为AS内部的转发端口由内部路由协议决定
 - 目的地址为AS外部的转发端口由两种协议决定

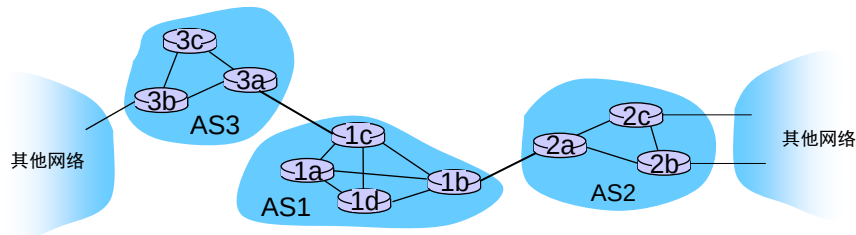
5-34

域间路由的任务

- AS1内部路由器收到去往外部的分组：
 - 要决定发给哪个网关路由器

域间路由算法的任务：

1. 知道AS1通过AS2和AS3可以去往哪些网络
2. 广播告诉内部路由器



5-35

内部路由协议

- 常用：
 - RIP: Routing Information Protocol
 - OSPF: Open Shortest Path First (IS-IS, 类似)
 - IGRP: Interior Gateway Routing Protocol (Cisco专用)

5-36

OSPF (Open Shortest Path First)

- 使用LS算法
 - 在AS内广播链路状态（OSPF报文封装到IP）
 - 各节点知道网络拓扑
 - Dijkstra计算转发表
- **IS-IS协议**：等同于OSPF

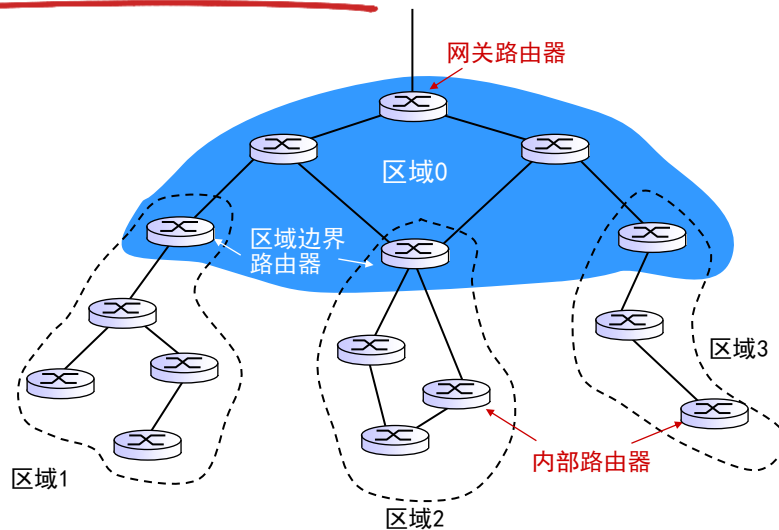
5-37

OSPF的特点

- **安全**: 报文可鉴别，身份，加密
- **允许相同代价路径** (RIP只能一条)
- **支持多种链路代价**，支持ToS
- **支持多播**
 - MOSPF与OSPF使用相同拓扑
- **支持层次结构**: AS分成若干区域，链路状态只在区域内广播。各路由器知道区域内拓扑，知道去区域外发给哪个区域边界路由器

5-38

层次化的OSPF



5-39

第5章: 网络层控制平面



5.1 简介

5.5 SDN控制平面

5.2 路由协议

5.6 ICMP协议

- 链路状态

5.7 SNMP协议

- 距离向量

5.3 自治系统内的路由:

OSPF

5.4 ISP之间的路由: BGP

5-40

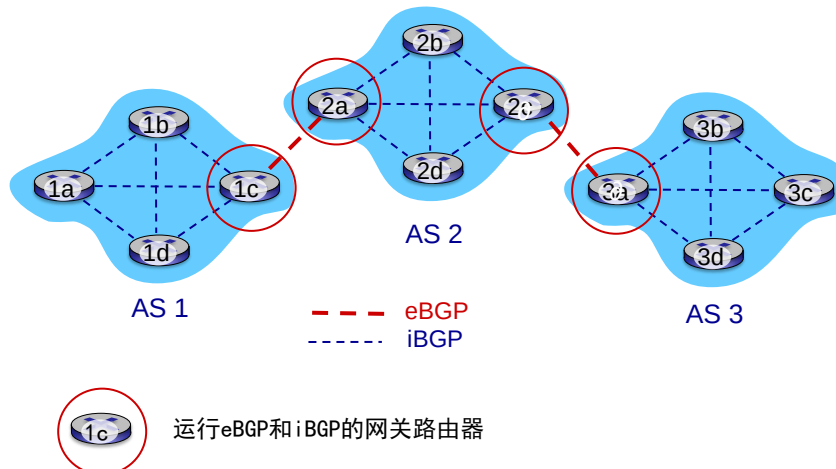
因特网的域间路由: BGP

- BGP (Border Gateway Protocol):
 - “将因特网ISP胶合在一起”
- 作用:
 - 外部BGP (eBGP): 向因特网通告内部子网的存在; 从邻居AS获取网络信息; 确定路由 (去某外网, 走哪个边界路由器和哪些AS)
 - 内部BGP (iBGP): 将路由告知内部路由器

5-41

eBGP与iBGP连接

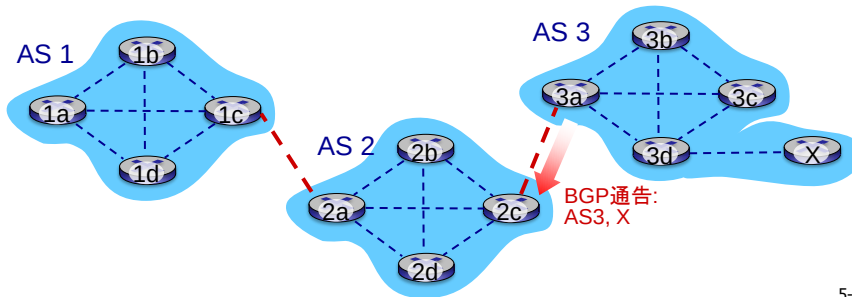
eBGP和iBGP都是TCP连接, 而不是物理链路



5-42

BGP

- BGP路由器在TCP连接上交换报文：
 - 应用层，基于管理员策略
 - 互相通告去往一些网络前缀的路径（路径向量协议）
- 3a告诉2c: **AS3, X**



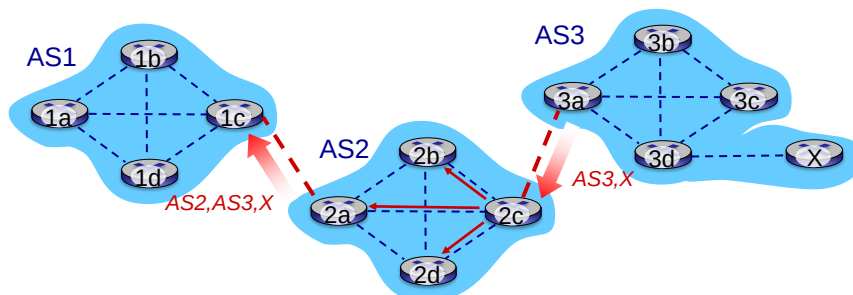
5-43

BGP

- **基于策略的路由:**（安全、政治等目的）
 - 收到路径通告，根据策略，决定接受或拒绝某路径
 - 根据策略，决定是否通告某邻居AS

5-44

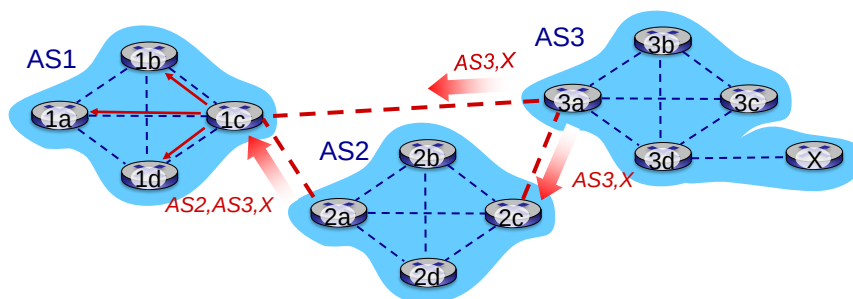
BGP



- 2c从3a 收到通告AS3, X（使用eBGP）
- 根据AS2的策略，2c接收此路径，并通告（使用iBGP）其内部路由器，包括2a
- 根据AS2的策略，2a 发送通告（使用eBGP）给AS2, AS3, X给1c

5-45

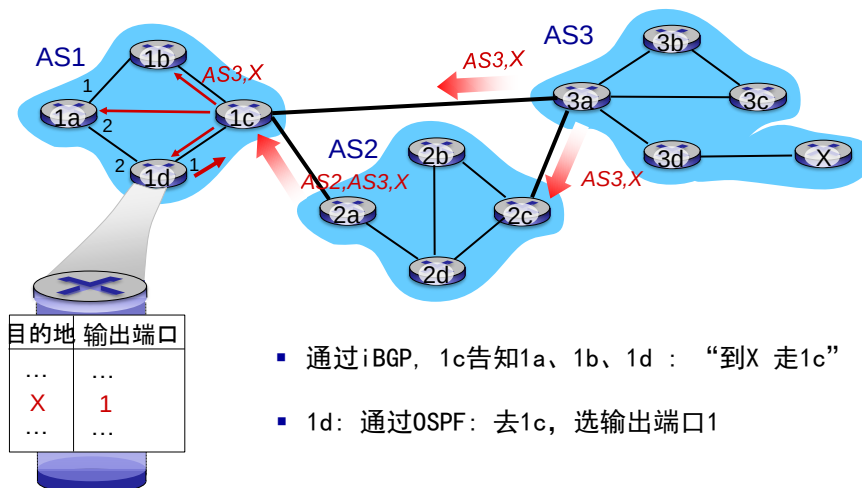
BGP



- 1c 从2a获知AS2, AS3, X
- 1c 从3a获知AS3, X
- 根据AS1的策略，1c只选择路径AS3, X，并通告其内部路由器（通过iBGP）

5-46

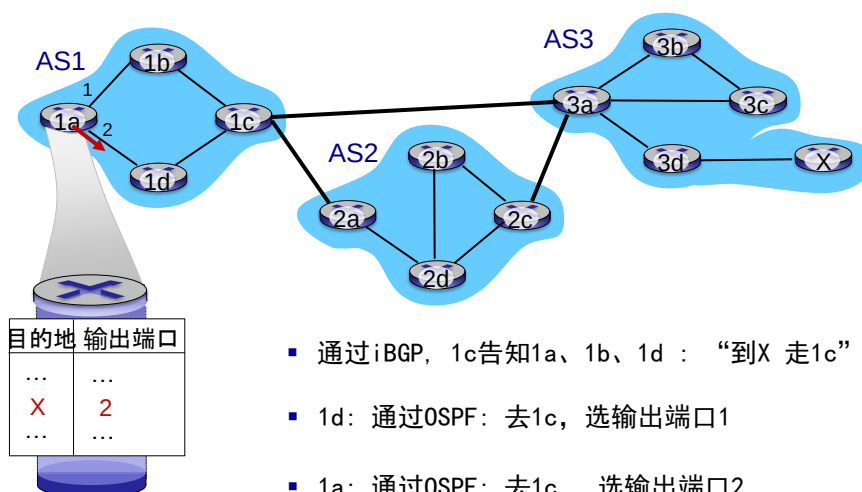
转发表



- 通过iBGP, 1c告知1a、1b、1d: “到X 走1c”
- 1d: 通过OSPF: 去1c, 选输出端口1

5-47

转发表



- 通过iBGP, 1c告知1a、1b、1d: “到X 走1c”
- 1d: 通过OSPF: 去1c, 选输出端口1
- 1a: 通过OSPF: 去1c, 选输出端口2

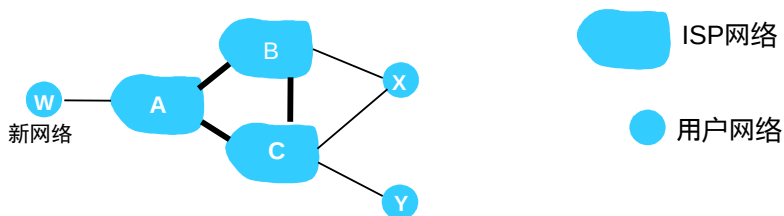
5-48

BGP的路由选择

1. 本地路由策略
2. 最短的AS路径
3. 热土豆路由
4. 其他

5-49

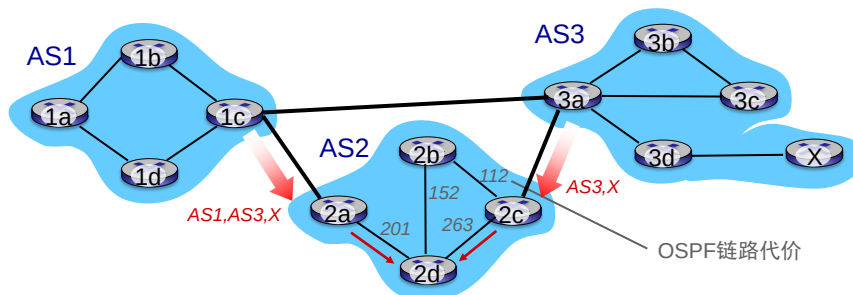
基于策略的路由



- A 通告路径Aw给B和C
- B 选择不通告路径BAw 给C:
 - B路由CBAw没有任何收益, 因为C,A,w不是B的客户
 - C不知道CBAw路径
- C 将使用CAw (不经过B) 到达w

5-50

热土豆路由



- 2d通过iBGP知道，可通过2a或2c去往X
- **热土豆路由**: 选择走2a!

尽可能快（严格说是更小的费用）地将分组送出其所在的AS，不关心外部AS到目的地的开销

5-51

域间路由 vs. 内部路由

策略:

- 域间路由: 策略路由: 允许谁使用其网络, 避开某些网络
- 内部路由: 相同管理者, 无需策略

性能:

- 域间路由: (政治、安全) 策略大于性能
- 内部路由: 性能至上

提升了扩展性:

- 层次路由减小转发表, 减少更新流量

5-52

第5章: 网络层控制平面



5.1 简介

5.2 路由协议

- 链路状态
- 距离向量

5.3 自治系统内的路由:

OSPF

5.4 ISP之间的路由: BGP

5.5 SDN控制平面

5.6 ICMP协议

5.7 SNMP协议

5-53

软件定义网络SDN

- 因特网网络层: 传统, 分布式每路由器控制
 - 路由器: 含分组交换硬件, 在专用操作系统 (e. g., Cisco IOS) 实现协议 (IP, RIP, OSPF, IS-IS, BGP)
 - 存在其它模块: 防火墙, 负载均衡, NAT, ...
- 2004~: 考虑逻辑集中式控制平面

5-54

软件定义网络SDN

为何采用集中式控制平面？

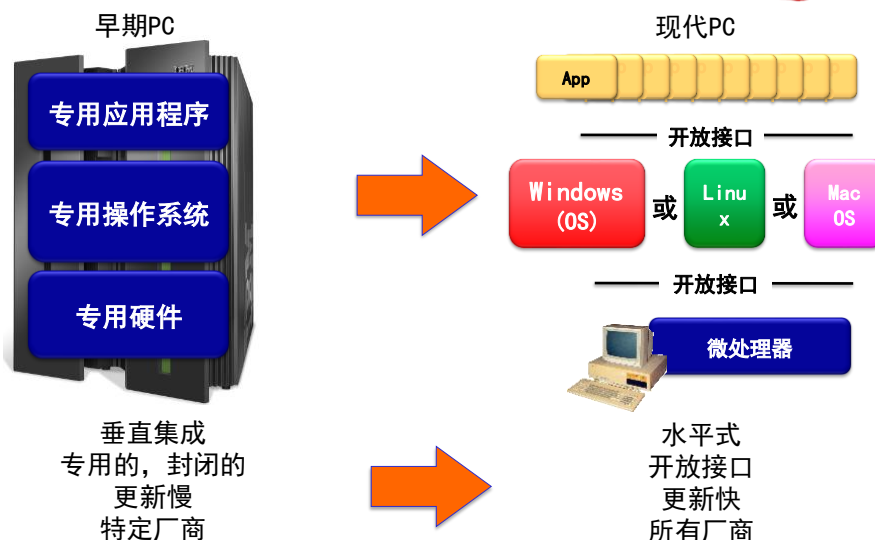
- 开放（非专用）实现
- 网管容易：避免路由器错误，强大的路由设置

软件定义网络SDN(Software Defined Network)：

一种新型网络创新架构（框架），其核心技术（标准）OpenFlow通过将网络设备的控制平面与数据平面分离，为核心网络及应用的创新提供了良好的平台。因为计算转发表并与路由器交互的控制器是用软件实现的，故网络是“软件定义”的。

5-55

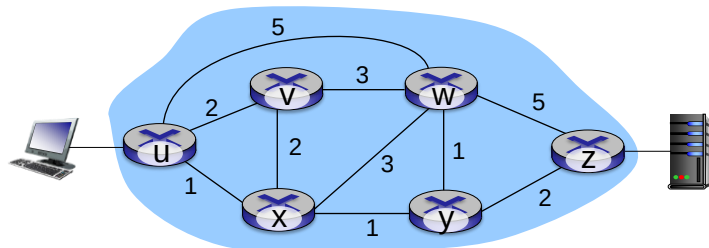
类比: PC的进化



PC: 专用->开放
因特网网络层: 每路由器控制->SDN

5-56

路由器控制的问题

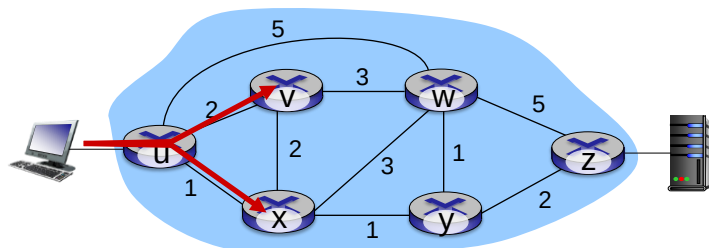


Q: 如何指定路径 $uvwz$?

A: 修改链路代价，让路由协议计算获得（或使用新的协议）！

5-57

路由器控制的问题

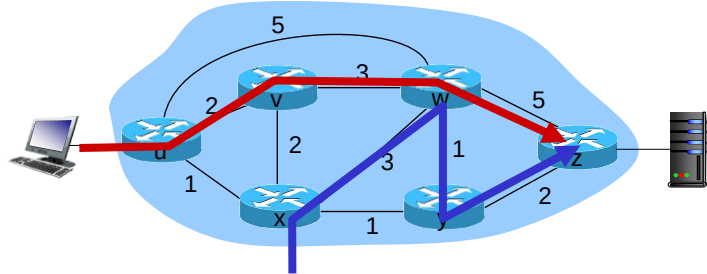


Q: 如何实现 u 到 z 走两条路径 $uvwz$ 和 $uxyz$?

A: 无法做到（或需要新的协议）！

5-58

路由器控制的问题

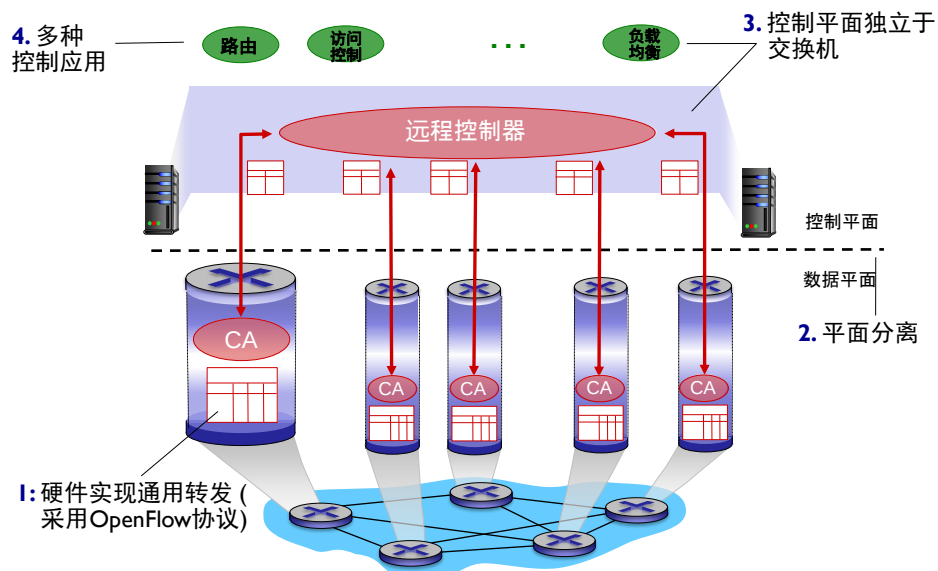


Q: 如何实现w区别传递红色和蓝色分组?

A: 无法做到 (基于目的地址的转发) !

5-59

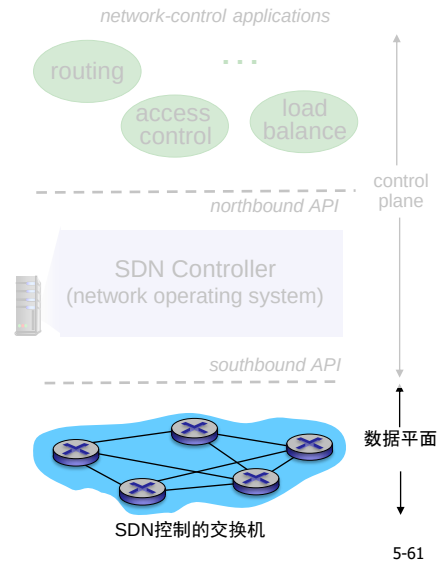
软件定义网络SDN



5-60

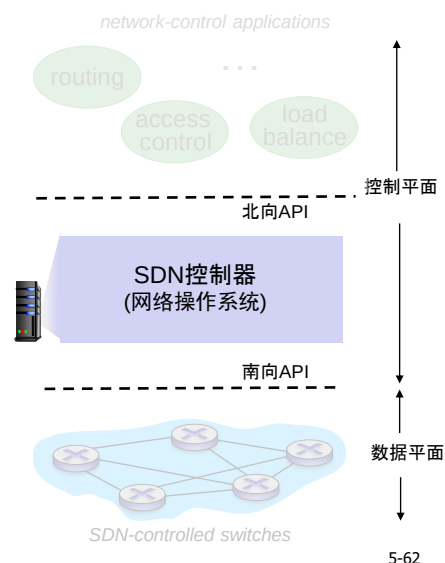
SDN: 数据平面交换机

- 硬件实现通用转发
- 控制器计算和分发流表
- 采用协议 (e. g., OpenFlow) 与控制器交互

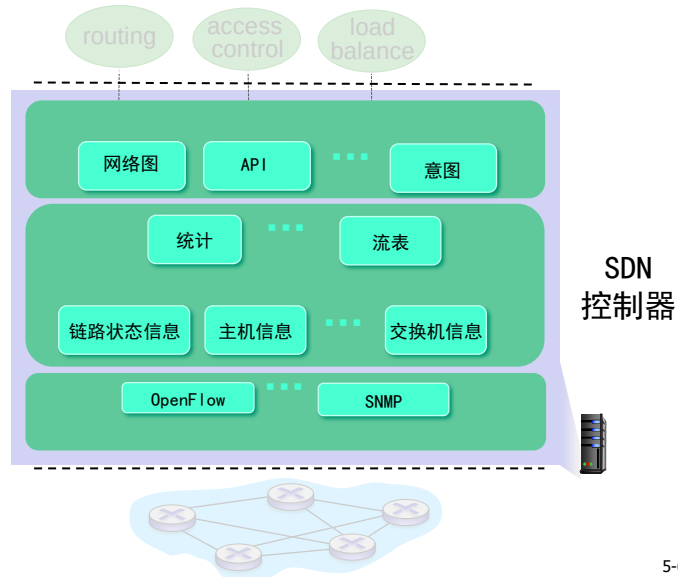


SDN: 控制器

- 维护网络信息
- 北向API与控制应用交互
- 南向API与交换机交互
- 分布式实现：更好的性能、扩展性、容错性、可靠性



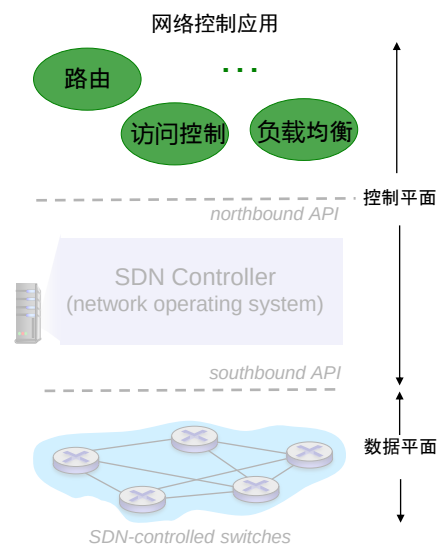
SDN控制器



5-63

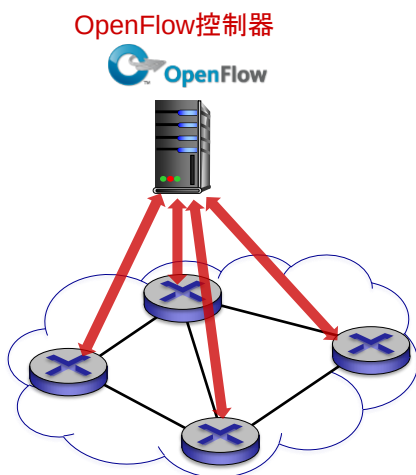
SDN: 网络控制应用

- 可由第三方提供



5-64

OpenFlow协议

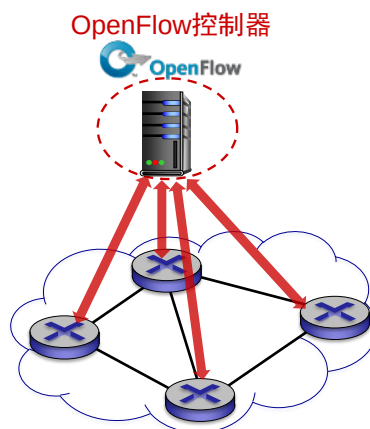


- 运行于SDN控制器和交换机之间
- 基于TCP
 - 报文加密（可选）

5-65

OpenFlow报文: 控制器到交换机

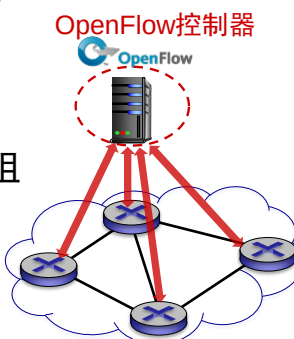
- **读状态**: 控制器收集交换机状态
- **配置**: 控制器查询并设置交换机参数
- **修改状态**: 增加、删除、修改流表项
- **出分组**: 控制器向特定交换机端口发送分组



5-66

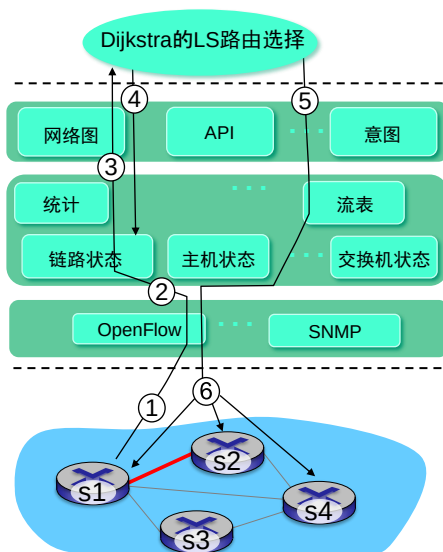
OpenFlow报文: 交换机到控制器

- **流删除**: 通知已删除一个流表项
- **端口状态**: 通告端口状态变化
- **入分组**: 交换机向控制器发送分组
(例如: 未匹配成功的分组、匹配动作要求发给控制器的分组)



5-67

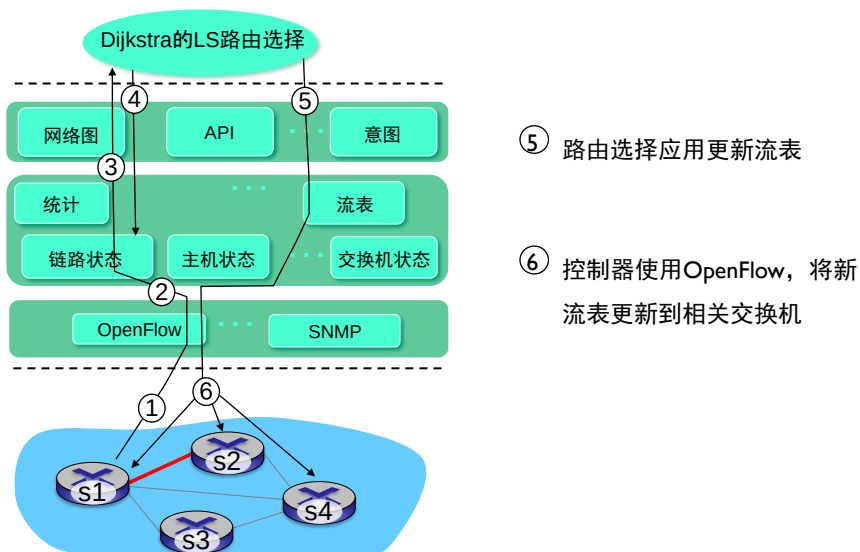
SDN交互的例子



- ① S1发现链路变化, 使用OpenFlow的端口状态报文通知控制器
- ② 控制器收到后, 更新链路状态信息
- ③ 路由选择应用已事先注册: 当链路状态变化, 得到通告
- ④ 路由选择应用访问网络图和链路状态, 计算新路由

5-68

SDN交互的例子



5-69

第5章: 网络层控制平面



5.1 简介

5.2 路由协议

- 链路状态
- 距离向量

5.3 自治系统内的路由:

OSPF

5.4 ISP之间的路由: BGP

5.5 SDN控制平面

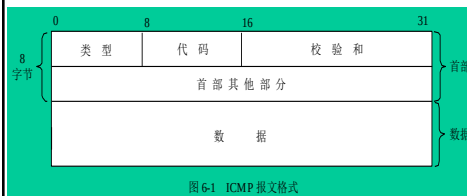
5.6 ICMP协议

5.7 SNMP协议

5-70

ICMP

- 网络层，IP之上
- 传送网络层控制信息，如
 - 回应请求与应答（ping、tracert命令）
 - 错误报告：不可达信息，路由器信息
- ICMP报文主要字段：类型，代码



类型	代码	描述
0	0	回应应答 (ping)
3	0	目的网络不可达
3	1	目的主机不可达
3	2	目的协议不可达
3	3	目的端口不可达
4	0	源抑制 (拥塞控制, 很少用, TCP有拥塞控制)
8	0	回应请求 (ping)
9	0	路由器通告
10	0	路由器发现
11	0	TTL过期
12	0	IP首部损坏

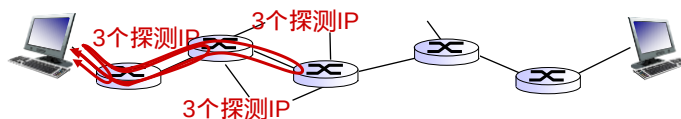
5-71

Tracert命令与ICMP

- 源主机向目的机发送一系列IP分组（携带端口不可达的UDP数据报）
 - 第一组3个，TTL =1
 - 第二组3个，TTL=2, ...
- 当第n组到达第n个路由器：
 - 丢弃分组，返回ICMP TTL过期报文（类型11，代码0）
 - 包含路由器地址
- 源主机收到后，计算RTT

停止规则：

- 到达目的机：目的机返回ICMP目的端口不可达报文（类型3，端口3）
- 源主机主动终止



5-72

第5章: 网络层控制平面



5.1 简介

5.5 SDN控制平面

5.2 路由协议

5.6 ICMP协议

- 链路状态

5.7 SNMP协议

- 距离向量

5.3 自治系统内的路由:

OSPF

5.4 ISP之间的路由: BGP

5-73

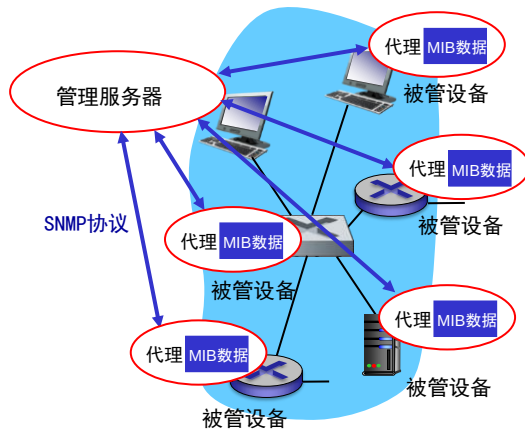
什么是网络管理?

- 为什么需要网络管理?

网络管理: 部署与协调硬件、软件和管理人员, 来监视、测试、调查、配置、分析、评价和控制网络及其部件, 达到以合理代价实现实时、高效、不同服务质量的网络通信。

5-74

因特网网络管理的框架



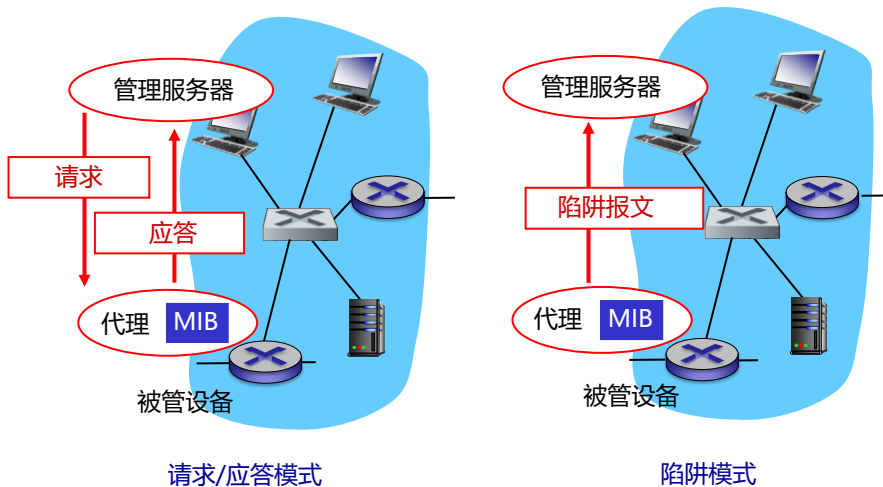
C/S架构:

- 管理服务器;
- 代理: 被管设备中被管对象数据存放在管理信息库 (MIB)
- SNMP协议

5-75

SNMP协议

应用层协议，基于UDP，有两种通信模式：



5-76

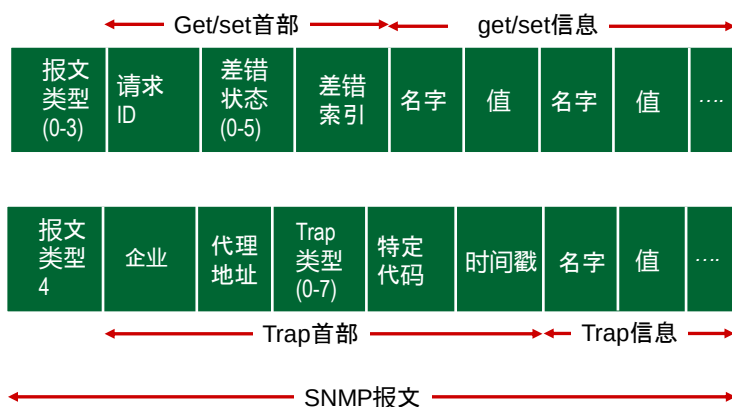
SNMP报文

类型	报文	功能
0	GetRequest	服务器-代理：获取一个对象、下一个对象和一批对象的数据
1	GetNextRequest	
	GetBulkRequest	
	InformRequest	服务器-服务器：传递MIB数据
2	SetRequest	服务器-代理：设置MIB数据
3	Response	代理-服务器：对请求的响应
4	Trap	代理-服务器：发生特定事件，通知服务器

SDN控制器使用SNMP监视与控制交换机

5-77

SNMP报文格式



5-78